THM | CAMPUS GIESSEN | MNI
TECHNISCHE HOCHSCHULE MITTELHESSEN
Mathematik, Naturwissenschaften und Informatik

Anfrageoptimierung – Datenbankbetrieb

Fortgeschrittene Datenbank-technologien

UNIVERSITY OF APPLIED SCIENCES
Marc Paulus

Begrüßung

- Name: Marc Paulus
- Bachelor Wirtschaftsinformatik
- Marc.Paulus@mnd.thm.de
- DevOps Engineer bei Kreuzwerker
- Erfahrungen im Cloud Computing und Betrieb von Kubernetes Plattformen

Gliederung

1. Datenbankanfrageoptimierung Warum?
2. Datenbankabfrageoptimierung Datenbanksystem
3. Datenbankabfrageoptimierung Organisatorisch
4. Datenbankabfrageoptimierung Queries

Datenbankabfrageoptimierung oft verdrängt?

- Datenbanken anfangs klein und schnell
- Wachsende Last durch:
  - Aufwendigere Queries/ Businessanforderungen
  - Mehr Nutzer und nutzende Services
  - Größere Datenbasis
- Datenbanken hochskalieren
- --> endlich und kostspielig
- Lösung: Effizienz durch Abfrageoptimierung

Schlechte Datenbankperfomance - Gründe

- Schlechtes Datenbankdesign
- Schlechter Datenbank fit (Exkurs)
- wachsende Datenmengen
- Schlechtes Abfragedesign
- Korrellierte subselects
- Schlechte Datenbankkonfiguration

Exkurs Unterschiede Postgresql vs. Mysql

- PostgreSQL
  - ORDBMS
  - Unterstützt viele verschiedene Datentypen und Formen(Arrays, Json, etc.)
  - M:N Beziehungen kompakt abbildbar
  - ACID compliant
  - Tendenziell besser geeignet für write heavy use cases
  - Spezialanforderungen entscheiden
- MySQL
  - RDBMS
  - Viele Umsetzungen über extra Tabellen und Joins
  - Bedingt acid compliant
  - Tendenziell besser geeignet für read heavy use cases
  - Spezialanforderungen entscheiden

Weitere Informationen:
https://www.ionos.de/digitalguide/server/knowhow/postgresql-vs-mysql/

Optimierung bei der Organisation der Datenbank

- Ressourcen
- Datenbanksystemauswahl
- Fixed Length fields*
- Korrekte Datenbanktypen
- **Intelligente Indexierung**
- **Materialized Views**
- **Caching**
- **Tabellendesign**
- Partitionierung

Datenbankabfrageoptimierung hat ein Limit bis zu welchem Grad es sinnvoll ist. Ab einem Gewissen Punkt werden mehr Ressourcen benötigt

Manche Datenbankmanagementsysteme sind für manche Aufgaben besser geeignet

*Immer abhängig vom jeweiligen Datenbanksystem ob Einstellungen nötig

Operationen und Abfragen können besser intern verarbeitet werden wenn die Datentypen möglichst eingegrenzt sind. Das betrifft eine richtige Typisierung und auch eine Beschränkung auf ein Minimum der Speicherverwendung bsp. (Short--> Int)

Die weiteren Punkte folgen auf eigenen Folien.

Weitere Informationen mysql optimierung https://www.linux-magazin.de/ausgaben/2003/03/abfragen-beschleuniger/4/

Intelligente Indexierung

- Beschleunigen Abfrage durch Verweise
- Indexsetzung bei frequentierten Attributen
- Nachteile:
- verbraucht Festplattenspeicher
- Verlangsamt Schreibvorgänge

Die Funktionsweise eines Index lässt sich grundsätzlich mit einem Index in einem Buch vergleichen. Eine sortierte Liste an Verweisen mit denen man nicht mehr die ganze Tabelle durchsuchen muss sondern nach einem Muster schneller das Ergebnis finden kann.

Lesezugriff beschleunigen im Austausch gegen Speicher und Schreibgeschwindigkeit, daher sollte es nur bei wirklich häufig genutzten oder sehr langsamen Abfragen genutzt werden. Besonders dafür eignen sich Fremdschlüssel die für Joins häufig gebraucht werden oder Daten auf denen häufig gefiltert wird wie ein Alter. In Maßen eine sehr sinnvolle Optimierung für mangelnde Read Performance

Materialized Views

- Vorberechnete kleinere Pseudotabelle
- Für: häufig abgefragte, selten geänderte, rechenintensive Abfragen
- Erhöht Query Performance
- Erhöht Speicherkosten

Mittelweg zwischen views die Abfragen nur für den Nutzer vereinfachen und extra rendundanten Tabellen, die immer aktualisiert werden müssen.

Eine Materialized View muss immer noch erneuert werden, wenn aktuelle Ergebnisse nachgeholt werden sollen und verbraucht wie die meisten anderen Optimierungen Speicherplatz. Daher nur wenn die Query häufig genug verwendet wird und die Daten sich nicht regelmäßig ändern. Sinnvoll für Select Optimierungen.

Quelle und tiefergehende Informationen: https://docs.snowflake.com/en/user-guide/views-materialized

Caching

- *internes Caching*
- RAM der Datenbank, zb für Queries, Results, indizes
- *Externes Caching*
- Meistens Resultcaching
- in memory Datenbanken wie Redis
- Caching Strategien wie: Lazy Loading oder Write Through

Externes und Internes Caching ist möglich; internes Caching wird immer angewendet kann jedoch noch optimiert werden mit Erfahrung welche Komponenten mit mehr RAM mehr Leistung bringen können. Beim internen Caching werden zur Beschleunigung der Datenbank im Regelfall die Queries die Ergebnisse und die Indizes teillweise gespeichert.

Das externe Caching erfordert normalerweise zusätzliche Konfiguration von Anwenderseite um die Datenbankanfragen erst im Cache zu behandeln, dies kann aber je nach Konstellation beispielsweise bei Cloud Anbietern automatisch geschehen. Die Caching Datenbanken wie Redis kann man dabei noch mit verschiedenen Strategien befüllen lassen.

Lazy Loading: Daten werden erst im Cache abgefragt und bei einem Cache miss in den Cache gespeichert. Eignet sich gut für hauptsächlich lesende Applikationen da Aktualität nicht sichergestellt wird.

Write Through: Daten werden gleichzeitig in der Datenbank und im Cache aktualisiert. Kein Problem mit der Aktualität des Caches dafür aber mit dem Speicher des Caches und dadurch mehr Gedanken in Richtung Cache Invalidation um den Cache zu entlasten.

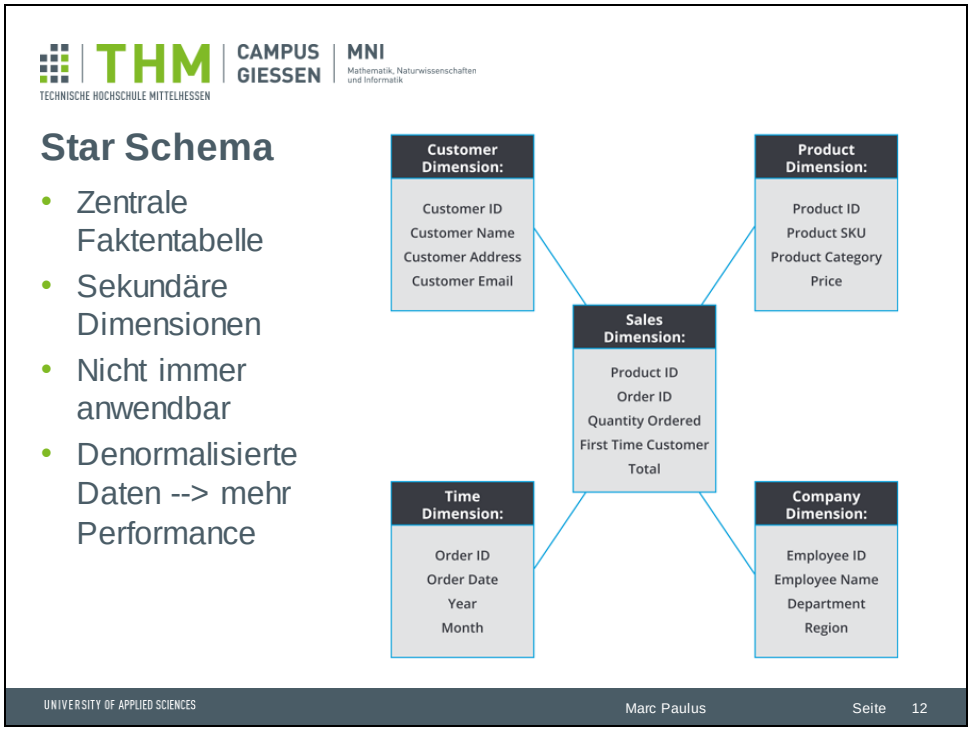
https://redis.com/de/losungen/anwendungsfaelle/caching/

https://severalnines.com/blog/overview-caching-postgresql/

Tabellendesign

- Star Schema
- Snowflake Schema
- Galaxy schema

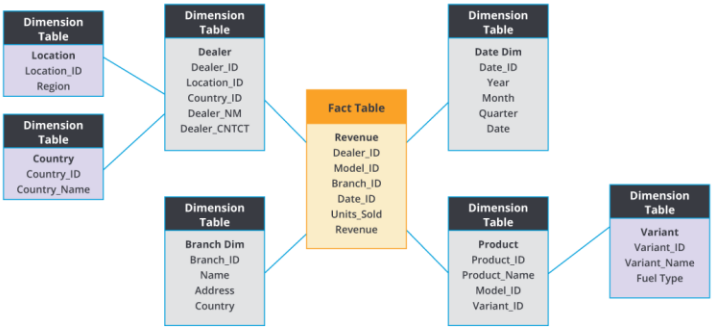
Neben der Normalform gibt es weitere Arten wie man die Daten in einer Tabelle organisieren kann. Diese eignen sich auch nicht für jeden Anwendungsfall jedoch ist besonders das Star Schema für unsere Zwecke von Interesse, da dieses Anfragen besonders beschleunigen kann und der Gedanke der Denormalisierung auch ohne Star Schema relevant sein kann. Snowflake und Galaxy Schema sind auch Schemata die helfen können Abfragen zu optimieren und sind nah mit dem Star Schema verwandt.



Die zentrale Faktentabelle enthält alle Fremdschlüssel der anderen Dimensionstabellen. Jeder Join muss dadurch immer über die Faktentabelle ablaufen. Das Schema kann nicht immer angewendet werden da nicht immer die Daten in passenden Beziehungen zueinander stehen. Für das Thema der Abfrageoptimierung ist jedoch interessant, das Daten wie hier in dem Beispiel das Total denormalisiert vorliegen weil sie häufig abgefragt werden und so keine komplizierten Joins notwendig sind.
Das Star Schema sorgt mit einer denormalisierten Datenbank für eine bessere Abfrageperformance

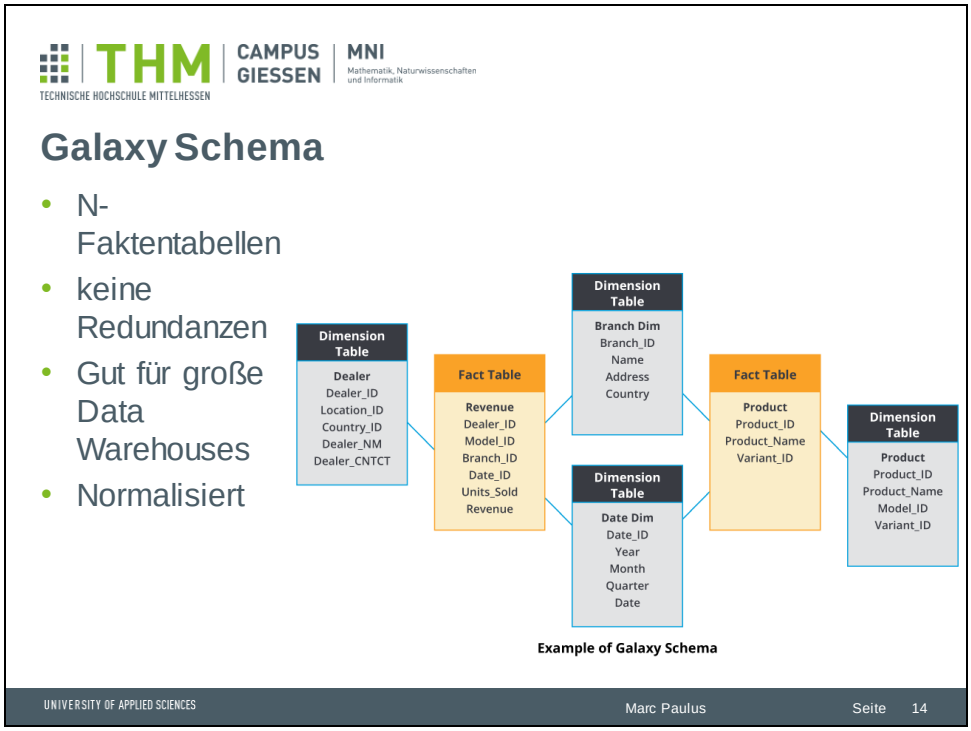
Snowflake Schema

- Daten normalisiert
- Geringere Abfrageperformance
- Dimensionen mit weiteren Dimensionen



Example of Snowflake Schema

Unterschied zum Star Schema können auch hier Dimensionstabellen noch Fremdschlüssel enthalten. Die Daten sind allgemein mehr normalisiert. Durch die fehlenden Redundanzen kann die Abfrageperformance geringer sein.



Im Gegensatz zu den vorigen Modellen kann es hierbei mehr als eine Faktentabelle mit vielen Fremdschlüsseln geben über die Verbindungen hergestellt werden.

Zudem ist es hoch normalisiert und optimiert für große Data Warehouses, da die Datenqualtität durch die normalisierung sehr hoch ist. Eine hohe Normalisierung und viele Tabellen kann hinderlich für die Optimierung sein kann aber auch förderlich sein, da so Schreibvorgänge einfacher sind und wenn richtig ausgeführt auch Joins eine akzeptable Geschwindigkeit haben. Die Entscheidung zwischen den Schemata hängt sehr viel von den zu speichernden Daten und den Anforderungen wie die Daten verarbeitet werden sollen.

Hinweis des Autors: meiner Meinung nach gehört der Wert Revenue nicht in die normlisierten Schemata, da sich dieser aus anderen Werten berechnen lässt.

Weitere Informationen und die Quelle der Bilder unter https://streamsets.com/blog/schemas-data-warehouses-star-galaxy-snowflake/

Quizfragen

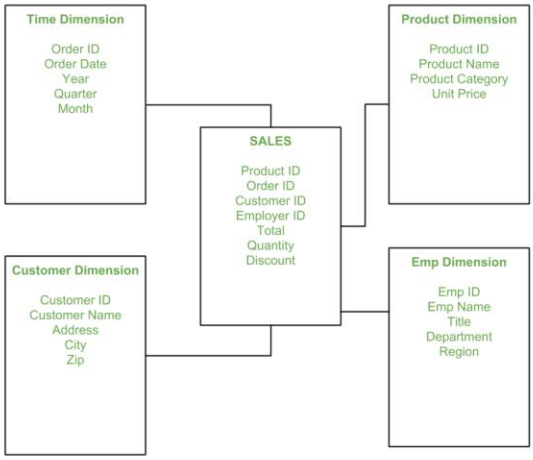
1. Welche Maßnahmen können organisatorisch vorgenommen werden um Datenbankabfragen zu beschleunigen? Nenne 4
2. Eine Abfrage zum Löschen dauert bereits für eine kleine Menge sehr lange was ist sinnvoll um hierbei eine Optimierung zu schaffen.
3. Welche Möglichkeit gibt es extern Abfragen zu beschleunigen? Die Daten ändern sich nicht allzu häufig und eine mögliche veraltete Antwort ist akzeptabel

Lösungen auf den weiteren Folien

Lösungen Quiz

1. Indexe, Materialized Views, Caching, Partitioning, DBMS wechseln, fixe Datentypenlängen, korrekte Datentypenauswahl, Ressourcen
2. Meistens liegt die mangelnde Perfomance bei Abfragen an großen Scans auf nicht indexierten Fremdschlüsseln mit denen Joins durchgeführt werden. Ohne Index erhöht sich die Abfragezeit deutlich
3. Caching; wenn zudem die vorhandenen Voraussetzungen gegeben sind kann lazy Loading verwendet werden.

Welches Schema?



Queryoptimierung

- Hinweis: Slow Query Logs
- Intelligente Abfragen
- Left/inner Join
- Minimaldaten
- korrelierte Subabfragen bedenken

- Bei mangelnder Datenbankperformance sollten zuerst die Slow Query Logs angeschaltet und untersucht werden. Diese enthalten meist die Abfragen, die die Datenbanken am meisten belasten.
- Generell sollte man diese Fragen intelligent umformulieren. Wie das genau optimiert werden kann ist auf viele verschiedene Weisen möglich, im Folgenden werden verschiedene Optimierungen formuliert die bedacht werden können.
- Select * distinct vermeiden, benutze limit um die Abfragemenge auf ein Minimum zu reduzieren zb. Um ein Beispiel der Datensätze zu sehen müssen nur ein paar Einträge geladen werden
- Immer mit den kleinen Tabellen anfangen
- Left und inner join crossjoins vorziehen
- Nach Möglichkeit korrelierte Subselects vermeiden da die Abfragezeit sich hierbei nicht additiv sondern multiplikativ verhält, statt M+N, MxN. Bei kleineren Abfragen nicht bedenklich aber wenn die Tabelle und die Unterfrage größer und aufwendiger werden

Übungen Anfrage-optimierung

Select Beispiel

- Zeige den Namen Code und Bevölkerung der Länder mit mehr als 1000000 Einwohner
- select name, code, population from country where population>1000000;
- Zeige dieselben Werte für das größte Land
- select name, code, population from country order by population desc Limit 1;
- Summiere die Bevölkerung der jeweiligen Kontinente
- select sum(population), continent from country group by continent;

Installation Docker Container

- Start Docker Desktop or similar tool
- https://hub.docker.com/r/aa8y/postgres-dataset/
- docker pull aa8y/postgres-dataset
- Tag/Name=world
- docker run -d --name pg-ds-<tag> aa8y/postgres-dataset:<tag>
- docker exec -it pg-ds-<tag> psql -d <db_name>

Auf der docker hub Seite stehen alle Informationen die man zur Nutzung der Postgres Datenbank Nutzung beachten sollte. Für unsere Übungen halten wir uns an die World Datenbank, da das Schema sehr einfach ist.

THM CAMPUS GIESSEN | MNI Mathematik, Naturwissenschaften und Informatik

TECHNISCHE HOCHSCHULE MITTELHESSEN

Datenbankabfrageoptimierung Wie?

```
marcpaulus — com.docker.cli • docker exec -it pg-ds-all psql -d world — 84×27
(1 row)

world=# explain select c.population,c.name,((select sum(population) from  city ci  w
here ci.countrycode=c.code)/c.population) from country c ;
                              QUERY PLAN
----------------------------------------------------------------------
 Seq Scan on country c  (cost=0.00..19855.15 rows=239 width=23)
   SubPlan 1
     ->  Aggregate  (cost=83.03..83.04 rows=1 width=8)
           ->  Seq Scan on city ci  (cost=0.00..82.99 rows=18 width=4)
                 Filter: (countrycode = c.code)
(5 rows)
```

```
world=# explain analyze select c.population,c.name,((select sum(population) from  ci
ty ci  where ci.countrycode=c.code)/c.population) from country c ;
                                                          QUERY PLAN

-----------------------------------------------------------------------------------------
-----------------------------------------------------
 Seq Scan on country c  (cost=0.00..8107.72 rows=239 width=23) (actual time=0.239..4
.277 rows=239 loops=1)
   SubPlan 1
     ->  Aggregate  (cost=33.88..33.89 rows=1 width=8) (actual time=0.017..0.017 row
s=1 loops=239)
           ->  Bitmap Heap Scan on city ci  (cost=4.42..33.84 rows=18 width=4) (actu
al time=0.009..0.012 rows=17 loops=239)
                 Recheck Cond: (countrycode = c.code)
                 Heap Blocks: exact=261
                 ->  Bitmap Index Scan on countrycode  (cost=0.00..4.42 rows=18 widt
h=0) (actual time=0.007..0.007 rows=17 loops=239)
                       Index Cond: (countrycode = c.code)
 Planning Time: 0.186 ms
 Execution Time: 4.354 ms
(10 rows)
```


Explain in PostgreSQL zeigt einen Plan an und eine Schätzung wie schnell die Query ist

Die Schätzungen werden dabei in einer künstlichen Einheit angegeben, die angibt wieviele 8kb Inhalte gelesen werden müssen.

Explain analyze wiederum zeigt die Statistiken an die bei der tatsächlichen Ausführungen anfallen. Die Anfrage wird auch tatsächlich ausgeführt, was bei deletes und updates auf jeden Fall beachtet werden sollte. Bei Selects ist es unbedenklich.

Was bei explain analyze jedoch auch beachtet werden sollte ist, dass sich die tatsächlichen Durchführungszeiten durch den Cache verändern können, daher sollte nicht nur ein Ergebnis zur Begutachtung als ausreichend genommen werden. Zudem sollte beachtet werden, dass Analyze die Abfrage verlangsamt.

Ein weiteren Hinweis der gegeben werden sollte ist das durch den Unterschied von Theorie und Praxis mit und ohne Analyze es auch dazu führen kann, dass Anfragen theoretisch besser wirken als sie sind und umgekehrt. Dies hat etwas mit den Berechnungsgrundlagen und Schätzungen zu tun. In Versuchen hat der Autor dies bei Abfragen mit Hashmaps festgestellt welche bei kleinen Tabellen gut funktionieren jedoch bei größeren Tabellen Hashmap Kollisionen als Störfaktor dazukommen.

Weitere Informationen für ein genaueres und besseres Benchmarking von Queries am Beispiel von Postgresql sind hier zu finden
https://www.tangramvision.com/blog/how-to-benchmark-postgresql-queries-well

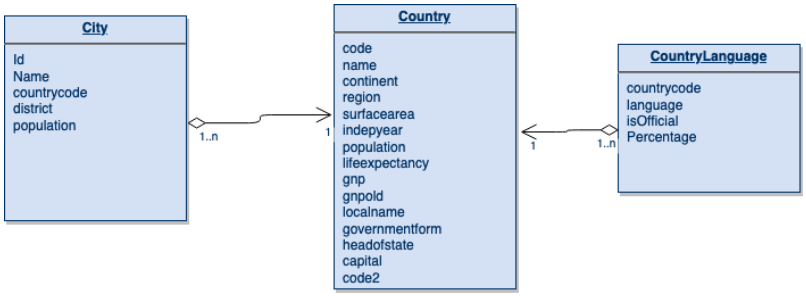
Postgresql Cheat Sheet

https://www.postgresqltutorial.com/postgresql-cheat-sheet/

\du show all users

\dt show all tables in the current database

\l list all databases in the server



Übungen Selects und mögliche Optimierungen

1. Ermittle das Land mit der größten Oberfläche. (Anfragehäufigkeit selten) Optimiere sinnvoll
2. Gebe die Städte zusammen mit der Stadtbevölkerung dem Namen des Landes und der Bevölkerung des Landes aus
3. Wieviel Prozent der Bevölkerung lebt in den Städten des Landes die in der Datenbank sind? Achte auf die Performance im Vergleich zu den anderen Queries vorher
4. Angenommen du willst Query nummer 3 Optimieren tue dies sinnvoll
5. Dein Auftraggeber interessiert sich besonders für die größten Städte eines Landes wie kannst du hier sinnvoll optimieren. Es ist damit zu rechnen, dass diese Städte in vielen Queries vorkommen.
6. EXTRA: Dein Auftraggeber möchte gerne wissen wieviel der Bevölkerung eines Landes in der jeweiligen größten Städt des jeweiligen Landes lebt. Benutze dafür gerne Ergebnisse aus 5.

Lösungen auf den folgenden Seiten

Übung Lösungen

- explain analyze select name, surfacearea from country order by surfacearea desc limit 1; oder
- explain analyze select name, surfacearea from country where surfacearea=(select max(surfacearea) from country) ;
- select c.population,ci.name,ci.population from country c inner join city ci on c.code=ci.countrycode;

Übung Lösungen

- select c.population,c.name,((select sum(population)*1.0 from city ci where ci.countrycode=c.code)/c.population*100) from country c;
- Oder:
- CREATE INDEX countrycode ON city (countrycode);

THM | CAMPUS GIESSEN | MNI Mathematik, Naturwissenschaften und Informatik
TECHNISCHE HOCHSCHULE MITTELHESSEN

Übung Lösung

- create materialized view big_city_per_country as Select id, name, countrycode, population from city c_ou where population=(select max(population) from city c_in where c_in.countrycode=c_ou.countrycode group by countrycode);
- explain analyze SELECT c.population, c.name, (total_population*1.0 / c.population * 100), bc.population, bc.population*1.0/total_population*100 FROM country c JOIN (SELECT countrycode, sum(population) AS total_population FROM city GROUP BY countrycode) AS city_population ON c.code = city_population.countrycode join big_city_per_country as bc on c.code=bc.countrycode;

Anfragen zur Perfektion zu optimieren ist schwer und langwierig. Es gilt hierbei einen Mittelweg zwischen Kosten für die Optimierung und Kosteneinsparung durch die Optimierung zu finden. Die Übungen und Lösungen hier sind Beispiele um den Umgang zu lernen sollte man nochmal auf zu optimierende Queries stoßen

Datenbankabfrageoptimierung Tools

- Explain mit Optionen
- interne automatische Optimierungen je Datenbank unterschiedlich
- Externe Tools wie EverSQL oder auch ChatGPT

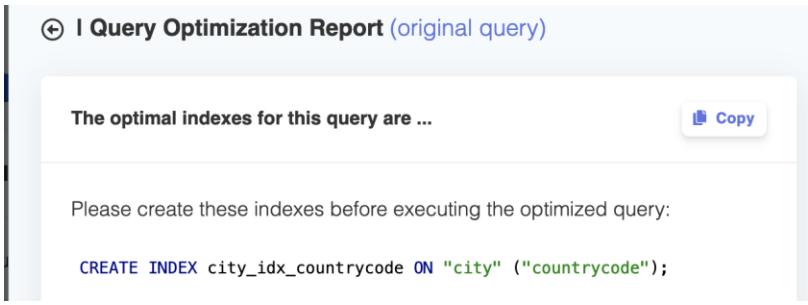
Das einfachste Werkzeug um Probleme initial zu analysieren ist ein einfacher explain um zu sehen, welche Abfragenteile besonders viel Ressourcen beanspruchen. Von dort kann man dann noch weitere Tools zur Hilfe ziehen oder die Query selbst optimieren

Je nach Datenbank werden intern verschiedene Optimierungen vorgenommen, diese unterscheiden sich je nach Datenbank und Version. Konkret geht es dabei um den Umgang mit Datentypen oder der Optimierung von Ablaufplänen.

Spezifische Hilfstools wie EverSQL als ein Beispiel von vielen oder das allgemeine Beispiel von ChatGPT sind Beispiele, wie man von Extern zusätzliche Ideen und Vorschläge zur Optimierung erhalten kann.

EverSQL

- Freie Testversion mit Upgrade für mehr Funktionen
- Empfehlungen basierend auf Schema
- Begrenzt intelligent (nicht alle Operatoren und Optimierungen)



Kostenlos mit beschränkter Funktion und Upgradefähig

EverSQL macht Empfehlungen basierend auf der Abfrage und dem Schema was es zusätzlich noch abfragt. Das macht die Empfehlungen im Vergleich zu ChatGPT etwas mehr basierend auf den Daten.

Unterstützt im Test keine Aggregationen und hat größtenteils nur Empfehlungen zu Indizes gemacht. Die Abfrage selbst wurde nicht verändert

THM CAMPUS GIESSEN | MNI Mathematik, Naturwissenschaften und Informatik
TECHNISCHE HOCHSCHULE MITTELHESSEN

Chat GPT

- Optionen zur Umschreibung ohne notwendige Schemakenntnis
- Auch mit Schema möglich
- Abfragen werden nur angeblich optimiert
- Verschlechterung im Test ~100 statt 30 in Theorie
- In der Praxis, durch einen kleinen Datensatz ~30% Verbesserung

sql Copy code

```sql
SELECT c.population, c.name, (total_population / c.population * 100)
FROM country c
JOIN (
    SELECT countrycode, sum(population) AS total_population
    FROM city
    GROUP BY countrycode
) AS city_population ON c.code = city_population.countrycode;
```

Diese optimierte Abfrage verwendet eine JOIN-Operation, um die Gesamtbevölkerung für jedes Land zu berechnen. Anstatt den Subquery für jedes Land separat auszuführen, wird eine Unterabfrage verwendet, um die Bevölkerungszahlen für jede Landescode (`countrycode`) zusammenzufassen. Die JOIN-Operation verknüpft dann die Tabelle `country` mit der aggregierten Tabelle `city_population` basierend auf dem Ländercode.

Bedenklich da ChatGPT immer eine Antwort liefert und diese nicht zwangsläufig besser ist als das ursprüngliche Ergebnis. Zudem wird die Query teilweise auch so umgeschrieben, dass sich die Ergebnismenge verändert.

Key Takeaways Datenbankabfrageoptimierung

- Slow Query Logs und Hinweise
- Vielseitig behandeln
- Tools sinnvoll aber Verständnis wichtiger
- Abfrageoptimierungskosten im Vergleich zu Arbeitsaufwand relativieren

Meist werden Probleme bei Datenbankanfragen durch Logs und Supportanfragen von Nutzern sichtbar. Diese sollten initial untersucht werden um mögliche Ansätze zu finden um die Datenbank zu entlasten. Das Problem kann und sollte hierbei von verschiedenen Optimierungspunkten betrachtet werden. Neben einem Umschreiben der Abfrage selbst können sich auch Indizes, Materialized Views, Cachekonfigurationen und weiteres anbieten.
Die Tools die es dafür auf dem Markt gibt sind sinnvoll und hilfreich müssen jedoch immer mit Vorsicht gebraucht werden, da diese auch nicht alle Optimierungen finden oder teilweise die Abfragen so umschreiben, dass das Ergebnis nicht mehr dasselbe ist. (Chatgpt)

Zu guter letzt ist es immer eine Aufwandsabschätzung, ob sich bei einer Abfrage eine Optimierung lohnt oder ob diese bereits ausreichend optimiert ist und ggf. mehr Ressourcen benötigt werden.